

S. N	Title	Submit Date	Sign
3.	To study different data types in MySQL.	2082/07/15	
4.	To implement the ALTER command in MySQL.	2082/07/15	
5.	To implement the Drop Command in MySQL.	2082/07/15	
6	To query the relational database in order to retrieve the required information by using the join operations.	2082/07/15	
7	To Create database schema and run SQL queries to retrieve the information as instructed.	2082/07/15	
8	To Create Employee Database Schema and Retrieve Information using SQL Aggregate Queries	2082/07/15	
9	TO create database schema and run the different UPDATE, DELETE and ALTER queries using MySQL.	2082/07/15	
10	Creating views in mysql.	2082/07/15	

Experiment No: 3

TITLE: TO STUDY DIFFERENT DATA TYPES IN MYSQL

1. Objectives

- 1.1 To understand the various data types supported by MySQL.
- 1.2 To learn how to use appropriate data types for storing different types of data.
- 1.3 To implement a table using multiple data types and observe their behavior.

2. THEORY

Data types are crucial in MySQL as they determine the kind of values a column can hold, which affects how MySQL stores and processes the data. Choosing the right data type helps optimize storage space, ensures data integrity, and improves performance. MySQL categorizes data types into three main groups: **numeric**, **string**, and **date and time**.

1. Numeric Data Types

Numeric data types are used to store numerical values, including integers, floating-point numbers, and fixed-point numbers.

- **Integer Types**
 - TINYINT: Used for very small integers. It has a signed range of -128 to 127 and an unsigned range of 0 to 255.
 - SMALLINT: A small integer. It has a signed range of -32,768 to 32,767 and an unsigned range of 0 to 65,535.
 - MEDIUMINT: A medium-sized integer. It has a signed range of -8,388,608 to 8,388,607 and an unsigned range of 0 to 16,777,215.
 - INT (or INTEGER): A standard integer. It has a signed range of -2,147,483,648 to 2,147,483,647 and an unsigned range of 0 to 4,294,967,295.
 - BIGINT: Used for very large integers. It has a signed range of -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 and an unsigned range of 0 to 18,446,744,073,709,551,615.
- **Fixed-Point Types**
 - DECIMAL (or NUMERIC): Used for storing precise numerical values where the exact decimal representation is needed, such as for financial data. It's declared with a precision and a scale, for example, DECIMAL(5, 2) can store a number with up to 5 digits in total, with 2 digits after the decimal point.
- **Floating-Point Types**
 - FLOAT: A single-precision floating-point number.
 - DOUBLE: A double-precision floating-point number. FLOAT and DOUBLE are used for storing approximate numeric values.

2. String Data Types

String data types are used to store text, binary data, and other character-based information.

- **Character Types**
 - CHAR: A fixed-length string. If you declare CHAR(10), it will always store 10 characters, padding with spaces if the input is shorter.
 - VARCHAR: A variable-length string. If you declare VARCHAR(255), it will store up to 255 characters, using only the space needed for the actual data.
- **Binary String Types**
 - BINARY: Similar to CHAR, but stores binary byte strings. It's fixed-length.
 - VARBINARY: Similar to VARCHAR, but stores binary byte strings. It's variable-length.
- **Large Text Types**
 - TINYTEXT: A very small text column with a maximum length of 255 characters.
 - TEXT: A small text column with a maximum length of 65,535 characters.
 - MEDIUMTEXT: A medium-sized text column with a maximum length of 16,777,215 characters.
 - LONGTEXT: A large text column with a maximum length of 4,294,967,295 characters.
- **Other String Types**
 - ENUM: An enumeration, which allows you to define a list of possible string values. For example, ENUM('male', 'female', 'other'). This is great for enforcing data integrity.
 - SET: A set, which allows you to store multiple string values from a predefined list. For example, SET('A', 'B', 'C') could store 'A', 'B', 'C', or any combination.
 -

3. Date and Time Data Types

Date and time data types are used to store temporal values.

- DATE: Stores a date in 'YYYY-MM-DD' format.
- TIME: Stores a time in 'HH:MM:SS' format.
- DATETIME: Stores both a date and time in 'YYYY-MM-DD HH:MM:SS' format. This type is not timezone-aware.
- TIMESTAMP: Stores a date and time in 'YYYY-MM-DD HH:MM:SS' format. This type is **timezone-aware**, meaning the value is stored in Coordinated Universal Time (UTC) and converted to the session's timezone upon retrieval. It's often used for tracking creation or modification times.
- YEAR: Stores a year in either 2-digit or 4-digit format.

3. DEMONSTRATION

Given below the table with its datatypes used.

Department Table :

- INT → for unique department ID.
- VARCHAR(100) → for department name and location (variable length strings).
- PRIMARY KEY → ensures each department has a unique ID.

```
MariaDB [maheshudas]> desc department
-> ;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dept_id    | int(11)       | NO   | PRI | NULL    |       |
| dept_name  | varchar(100)  | NO   |     | NULL    |       |
| location   | varchar(100)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.057 sec)
```

Student Table:

- INT → for student ID and department ID.
- BIGINT → for registration number and contact number (large numeric values).
- VARCHAR(100) → for storing full name.
- ENUM → for fixed gender options (Male, Female, Other).
- DATE → for storing date of birth.
- TEXT → for longer addresses.
- DECIMAL(3,2) → for GPA with precision (e.g., 3.75).
- FOREIGN KEY → links student to department.

```
MariaDB [maheshudas]> desc student;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stud_id    | int(11)       | NO   | PRI | NULL    |       |
| reg_no     | bigint(20)    | YES  | UNI | NULL    |       |
| full_name  | varchar(100)  | NO   |     | NULL    |       |
| gender     | enum('Male', 'Female', 'Other') | YES  |     | NULL    |       |
| dob        | date          | YES  |     | NULL    |       |
| contact    | bigint(20)    | YES  |     | NULL    |       |
| address    | text          | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
| gpa        | decimal(3,2)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

Course Table:

- INT → for course ID and department ID.
- VARCHAR(100) → for course name.
- TINYINT → for credit hours (small integer values).
- FOREIGN KEY → links course to department.

```
MariaDB [maheshudas]> describe course;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| course_id  | int(11)       | NO   | PRI | NULL    |       |
| course_name | varchar(100)  | NO   |     | NULL    |       |
| credit_hours | tinyint(4)    | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

4. RESULT AND DISCUSSION

The tables were successfully created using various MySQL data types such as INT, BIGINT, VARCHAR, TEXT, DECIMAL, ENUM, DATE, DATETIME, and CHAR. The student table demonstrated realistic fields such as registration number (BIGINT), GPA (DECIMAL), gender (ENUM), and address (TEXT). This implementation showed how different data types can be used together to create a practical and well-structured relational database design. Data types for optimized storage and accurate representation of data.

5. CONCLUSION

This experiment provided practical exposure to MySQL data types, emphasizing their importance in database design. Proper selection of numeric, string, and date/time types ensures efficiency, integrity, and accurate data manipulation. Through this exercise, students learned how to implement and work with multiple data types in real-world scenarios.

Experiment No: 4

TITLE: TO IMPLEMENT THE ALTER COMMAND IN MYSQL.

1. OBJECTIVES

- 1.1 To study the purpose and usage of the ALTER command in MySQL.
- 1.2 To learn how to modify the schema of an existing table using ALTER.
- 1.3 To implement actions such as adding, dropping, modifying columns, changing table names.

2. THEORY

In MySQL, the ALTER command is used to modify the structure of an existing table without dropping or recreating it. It provides flexibility to update the schema as database requirements evolve. Using ALTER, one can add new columns, delete unnecessary ones, rename existing columns, or modify their data type and constraints. It can also be used to rename a table entirely, add or remove primary keys, and perform multiple schema changes in a single statement.

The ALTER command is especially useful in real-world database systems where requirements may change frequently. For instance, a company may need to add a new contact column for employees, change the length of a string column to store more data, or rename tables for clarity. Instead of recreating the table, the ALTER command allows changes while preserving existing data.

The ALTER commands are given bellow:

1. Adding a Column

This command is used to add a new column to an existing table. You must specify the name of the new column and its data type.

- Syntax: ALTER TABLE table_name ADD COLUMN column_name datatype;
- Explanation: ALTER TABLE initiates the modification. ADD COLUMN is the action, followed by the new column's name and its datatype.
- Example: ALTER TABLE customers ADD COLUMN email VARCHAR(255);
 - This adds an email column to the customers table. The data type VARCHAR(255) allows it to store variable-length strings up to 255 characters.

2. Removing a Column

This command is used to remove an unwanted column from a table. This action is permanent and will delete all data stored in that column.

- Syntax: ALTER TABLE table_name DROP COLUMN column_name;
- Explanation: DROP COLUMN is the command to delete a column. You simply provide the name of the column you want to remove.

- Example: ALTER TABLE employees DROP COLUMN phone_number;
 - This command removes the phone_number column from the employees table.

3. Modifying a Column

This command is used to change the properties of an existing column, such as its data type or size.

- Syntax: ALTER TABLE table_name MODIFY COLUMN column_name new_datatype;
- Explanation: MODIFY COLUMN tells MySQL to alter the column's definition. You specify the column_name and its new_datatype.
- Example: ALTER TABLE products MODIFY COLUMN price DECIMAL(10, 2);
 - This changes the price column's data type to DECIMAL(10, 2), which is better for storing precise monetary values.

4. Changing a Column's Name and Properties

This is a more powerful command used to rename a column and, optionally, change its properties at the same time.

- Syntax: ALTER TABLE table_name CHANGE COLUMN old_column_name new_column_name new_datatype;
- Explanation: CHANGE COLUMN requires both the old_column_name and the new_column_name, followed by the new datatype.
- Example: ALTER TABLE users CHANGE COLUMN user_name username VARCHAR(50);
 - This command renames the user_name column to username and sets its new data type to VARCHAR(50).

5. Changing a Table's Name

This command is used to rename an entire table.

- Syntax: ALTER TABLE old_table_name RENAME TO new_table_name;
- Explanation: RENAME TO is the clause that handles the table name change. You simply provide the old_table_name and the new_table_name.
- Example: ALTER TABLE customer_info RENAME TO clients;
 - This changes the table name from customer_info to clients.

3. DEMONSTRATIONS

The base table:

```
MariaDB [maheshudas]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
# rows in set (0.068 sec)
```

1. Add a Column

```
MariaDB [maheshudas]> alter table employees add column phone varchar(15);
Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| phone      | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

2. Remove a Column

```
MariaDB [maheshudas]> alter table employees drop column phone;
Query OK, 0 rows affected (0.058 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

3. Modify a Column (Change Data Type or Constraints)

```
MariaDB [maheshudas]> alter table employees modify column phone varchar(20);
Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| phone      | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

4. Change Column Name and Properties

```
MariaDB [maheshudas]>alter table employees change phone contact_number varchar(20);

Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc employees;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | YES | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

5. Change Table Name

```
MariaDB [maheshudas]>
MariaDB [maheshudas]> alter table employees rename to staff;
Query OK, 0 rows affected (0.046 sec)
```

6. Add a Primary Key

```
MariaDB [maheshudas]> alter table staff add primary key (emp_id);
Query OK, 0 rows affected (0.063 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]> desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | PRI | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

7. Drop Primary Key

```
MariaDB [maheshudas]>alter table staff drop primary key;
Query OK, 0 rows affected (0.079 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

8. Perform Multiple Actions in a Single Statement

```
MariaDB [maheshudas]>alter table staff add column age int, drop column contact_number;
Query OK, 0 rows affected (0.043 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| age | int(11) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.057 sec)
```

4. RESULT AND DISCUSSION

The experiment demonstrated how the ALTER command can be used to update the schema of an existing table. We successfully added new columns, dropped unnecessary ones, modified data types, changed column names, and even renamed the table. The use of constraints such as primary keys was also shown. This illustrates that ALTER is a versatile command for schema evolution in MySQL.

5. CONCLUSION

The ALTER command is one of the most powerful tools in SQL for managing changes in table structures. It allows database designers to adapt to changing requirements without losing data. This experiment highlighted various operations like adding/removing columns, modifying properties, and renaming tables. Understanding and applying ALTER properly ensures efficient and flexible database management.

Experiment No: 5

TITLE: TO IMPLEMENT THE DROP COMMAND IN MYSQL.

1. OBJECTIVES

- 1.1 To understand the use of the DROP command in MySQL.
- 1.2 To learn how to remove database objects like tables, databases, and constraints.
- 1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

The DROP command in MySQL is a Data Definition Language (DDL) statement used to **delete an entire database, table, or view**. It's a permanent and irreversible action that removes the object and all its associated data and structure from the database.

A. Drop database

This command is used to **delete an entire database**, including all its tables, views, and data. This action cannot be undone, so use it with extreme caution.

- **Syntax:** DROP DATABASE [IF EXISTS] database_name;
- **Explanation:** The IF EXISTS clause is optional but highly recommended. It prevents an error from occurring if you try to drop a database that doesn't exist.
- **Example:** DROP DATABASE IF EXISTS old_project;
 - This command will delete the database named old_project if it exists.

B. Drop table

This command is used to **delete a table** and all the data within it. This action is also permanent and should be used carefully.

- **Syntax:** DROP TABLE [IF EXISTS] table_name;
- **Explanation:** Similar to DROP DATABASE, the IF EXISTS clause prevents an error if the specified table doesn't exist.
- **Example:** DROP TABLE IF EXISTS temporary_users;
 - This command will delete the temporary_users table and all its rows.

3. DEMONSTRATIONS

```
MariaDB [(none)]> create database college;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]>
MariaDB [(none)]> use collage;
ERROR 1049 (42000): Unknown database 'collage'
MariaDB [(none)]> use college;
Database changed
```

Database creation :

Table creation:

```
MariaDB [college]> desc student ; desc course;
```

Field	Type	Null	Key	Default	Extra
stud_id	int(11)	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
age	int(11)	YES		NULL	
major	varchar(50)	YES		NULL	

4 rows in set (0.022 sec)

Field	Type	Null	Key	Default	Extra
course_id	int(11)	NO	PRI	NULL	
course_name	varchar(100)	YES		NULL	
credits	int(11)	YES		NULL	

3 rows in set (0.012 sec)

Drop table:

```
MariaDB [college]> drop table course;
Query OK, 0 rows affected (0.057 sec)

MariaDB [college]> drop table student;
Query OK, 0 rows affected (0.013 sec)

MariaDB [college]> desc student ; desc course;
ERROR 1146 (42S02): Table 'college.student' doesn't exist
ERROR 1146 (42S02): Table 'college.course' doesn't exist
```

Drop database:

```
MariaDB [(none)]>
MariaDB [(none)]> drop database college;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> show databases;
```

Database
3daysworkshop
ashishhlab2
ashishlab2
ashishsaud
e_store
information_schema
lab2student
mysql
performance_schema
phpmyadmin
test

11 rows in set (0.001 sec)

4. CONCLUSION

The DROP command is a powerful DDL statement in MySQL used to delete database objects permanently. While it is simple and effective, it should be used with caution as it leads to irreversible data and schema loss.

Experiment No: 6

TITLE: TO QUERY THE RELATIONAL DATABASE IN ORDER TO RETRIEVE THE REQUIRED INFORMATION BY USING THE JOIN OPERATIONS.

1. OBJECTIVES

- 1.1 To understand the use of the DROP command in MySQL.
- 1.2 To learn how to remove database objects like tables, databases, and constraints.
- 1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

JOIN operations in SQL are used to retrieve data from two or more tables by linking them together. This is a fundamental concept in relational databases, allowing you to create a single result set from fragmented data. The common link is usually a **foreign key** column that references the primary key of another table.

Types of JOINS

A. INNER JOIN

The INNER JOIN returns only the rows where the join condition is true in **both** tables. It's the most common type of join and is used to find the intersection of data between two tables.

- **Syntax:** SELECT columns FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** This query will only show rows where the value in table1.column_name is exactly the same as the value in table2.column_name.
- **Analogy:** Imagine you have a list of students and a list of courses. An INNER JOIN would give you a list of students who are **currently enrolled** in a course.

B. LEFT JOIN

The LEFT JOIN (or LEFT OUTER JOIN) returns **all rows from the left table** and the **matching rows from the right table**. If there is no match in the right table, the columns from that table will be filled with NULL.

- **Syntax:** SELECT columns FROM table1 LEFT JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** The table1 (the "left" table) is the one listed first in the query. The result set will contain every row from table1, whether it has a match in table2 or not.
- **Analogy:** If you want a list of every student in the school (left table) and the name of the course they're taking (right table), a LEFT JOIN would show all students, even those who aren't currently enrolled in any course.

C. RIGHT JOIN

The RIGHT JOIN (or RIGHT OUTER JOIN) is the opposite of LEFT JOIN. It returns **all rows from the right table** and the **matching rows from the left table**. Unmatched rows from the left table will have NULL values.

- **Syntax:** SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** The table2 (the "right" table) is the one listed after the RIGHT JOIN clause. The result will contain every row from table2, regardless of whether a match exists in table1.
- **Analogy:** Using the student and course example, a RIGHT JOIN would give you a list of every course offered (right table) and the names of the students enrolled in them. Courses with no students would still be on the list.

D. FULL JOIN

The FULL JOIN (or FULL OUTER JOIN) returns **all rows when there is a match in either the left or right table**. It essentially combines the results of both LEFT JOIN and RIGHT JOIN, showing all rows from both tables.

- **Syntax (Simulated in MySQL):** SELECT columns FROM table1 LEFT JOIN table2 ON table1.column = table2.column UNION SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column = table2.column WHERE table1.column IS NULL;
- **Explanation:** MySQL doesn't directly support FULL JOIN with a single keyword. The common method is to use a UNION of a LEFT JOIN and a RIGHT JOIN. The UNION operator combines the results of the two select statements.
- **Analogy:** A FULL JOIN would provide a complete list of every student **and** every course, regardless of whether a student is enrolled in a course or a course has any students.

3. DEMONSTRATIONS

Schema: **STUDENT(sid, name, address, contact)**
 BOOK(bid, title, author)
 BURROW(sid, bid)

```
MariaDB [maheshudas]> desc student; desc book; desc burrow;
```

Field	Type	Null	Key	Default	Extra
sid	int(11)	NO	PRI	NULL	
name	text	YES		NULL	
address	text	YES		NULL	
contact	text	YES		NULL	

4 rows in set (0.056 sec)

Field	Type	Null	Key	Default	Extra
bid	int(11)	NO	PRI	NULL	
title	text	YES		NULL	
author	text	YES		NULL	

3 rows in set (0.026 sec)

Field	Type	Null	Key	Default	Extra
sid	int(11)	YES	MUL	NULL	
bid	int(11)	YES	MUL	NULL	

2 rows in set (0.010 sec)

Queries:

a. All the information from student inner join burrow.

```
MariaDB [maheshudas]> SELECT * FROM student INNER JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209

9 rows in set (0.001 sec)

b. All the information from student left join burrow.

```
MariaDB [maheshudas]> SELECT * FROM student LEFT JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
107	ritu thapa	chitwan	9823456789	NULL	NULL
108	binod rai	illam	9801010101	NULL	NULL
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209
110	suman bhattarai	pokhara	9865432100	NULL	NULL

c. All the information from student right join burrow.

```
MariaDB [maheshudas]>SELECT * FROM student RIGHT JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209

```
9 rows in set (0.001 sec)
```

d. Name of the students burrowing at least one book.

```
MariaDB [maheshudas]> SELECT DISTINCT name FROM student INNER JOIN burrow ON student.sid=burrow.sid;
```

name
ashish saud
prashant dahal
sunita shrestha
prajwal bhandari
diya kc
manoj gurung
samikshya ghimire

```
7 rows in set (0.026 sec)
```

e. Title of all the books that are burrowed.

```
MariaDB [maheshudas]> SELECT DISTINCT title FROM book INNER JOIN burrow ON book.bid=burrow.bid;
```

title
the alchemist
siddhartha
the hobbit
a song of ice and fire
the great gatsby
pride and prejudice
moby dick
the lord of the rings
1984

```
9 rows in set (0.026 sec)
```

f. Name of the students burrowing no books.

```
MariaDB [maheshudas]> SELECT name FROM student LEFT JOIN burrow ON student.sid=burrow.sid WHERE bid is NULL;
```

name
ritu thapa
binod rai
suman bhattarai

```
3 rows in set (0.001 sec)
```

g. Title of all the books that are not burrowed yet.

```
MariaDB [maheshudas]> SELECT title FROM book LEFT JOIN
burrow ON book.bid = burrow.bid WHERE sid is NULL;
+-----+
| title |
+-----+
| to kill a mockingbird |
+-----+
1 row in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The queries executed successfully and produced the required results. The INNER JOIN query displayed students who have borrowed books along with their details. The LEFT JOIN retrieved all students, including those who have not borrowed any books, with NULL values in unmatched cases. Similarly, the RIGHT JOIN retrieved all borrowed book entries along with corresponding student details. Queries with DISTINCT ensured that repeated names or titles were not displayed multiple times. Finally, the queries using WHERE ... IS NULL helped identify students who have not borrowed any books and books that are still unborrowed. This demonstrated the importance of different join operations in extracting specific relationships from a relational database.

6. CONCLUSION

This experiment successfully demonstrated the use of different join operations in MySQL. By using INNER JOIN, LEFT JOIN, and RIGHT JOIN, along with filtering conditions, we were able to retrieve important information such as students borrowing books, students not borrowing books, borrowed books, and books that are still unborrowed. Thus, join operations are essential tools for querying multiple tables in relational databases effectively.